Dismiss

Join GitHub today

GitHub is home to over 36 million developers working together to host and review code, manage projects, and build software together.

Sign up

Branch: master

Find file Copy path

Convolutional_neural_network / convolutional_network_tutorial.ipynb

llSourcell Add files via upload

b643d31 on Jul 11, 2017

1 contributor

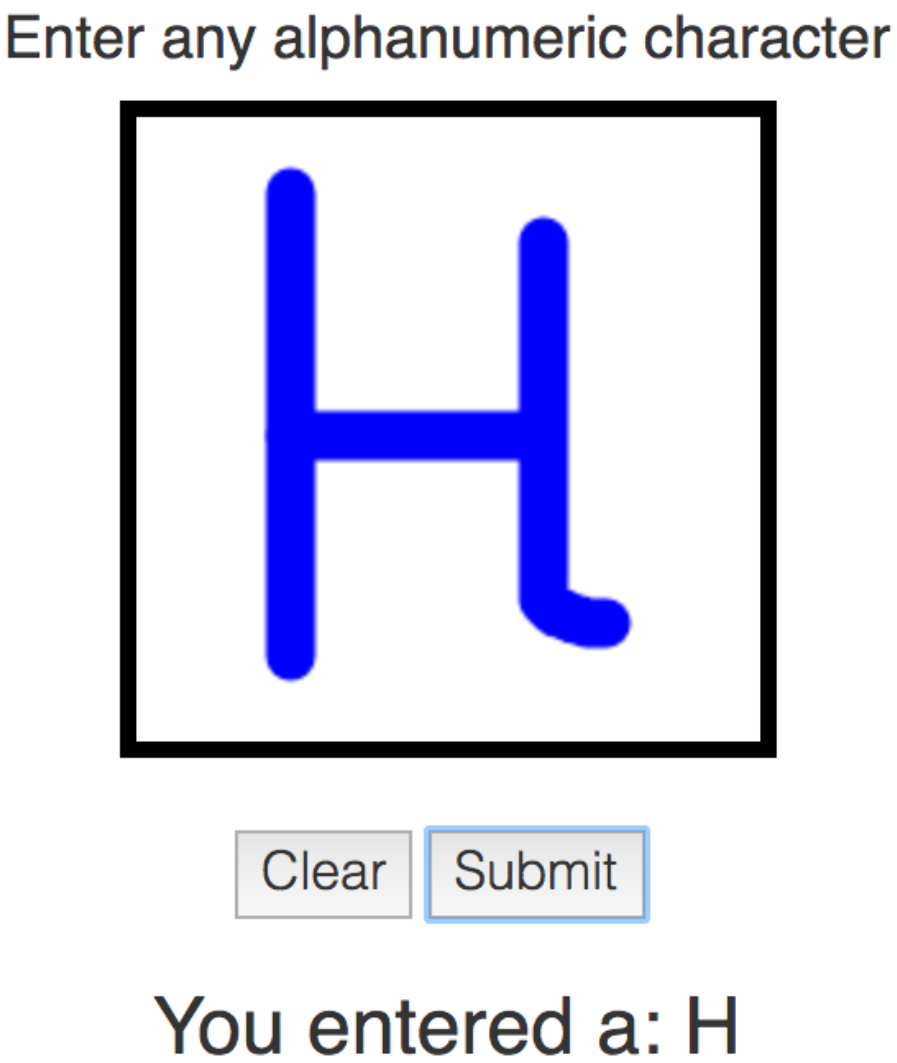
Raw Blame History

409 lines (408 sloc) 22 KB

Convolutional Network

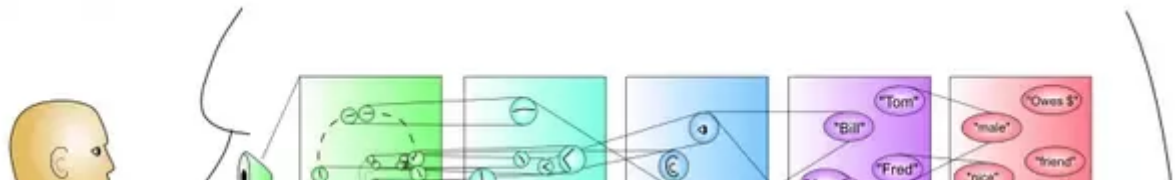
Demo

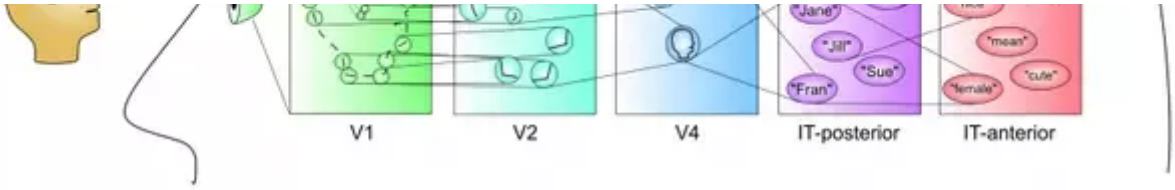
We're going to build a Convolutional Neural Network using just numpy capable of detecting any alphanumeric character that a user draws. It will all be wrapped into a Flask web app so you can play with it in the browser!



What inspired Convolutional Networks?

CNNs are biologically-inspired models inspired by research by D. H. Hubel and T. N. Wiesel. They proposed an explanation for the way in which mammals visually perceive the world around them using a layered architecture of neurons in the brain, and this in turn inspired engineers to attempt to develop similar pattern recognition mechanisms in computer vision.





In their hypothesis, within the visual cortex, complex functional responses generated by "complex cells" are constructed from more simplistic responses from "simple cells'.

For instances, simple cells would respond to oriented edges etc, while complex cells will also respond to oriented edges but with a degree of spatial invariance.

Receptive fields exist for cells, where a cell responds to a summation of inputs from other local cells.

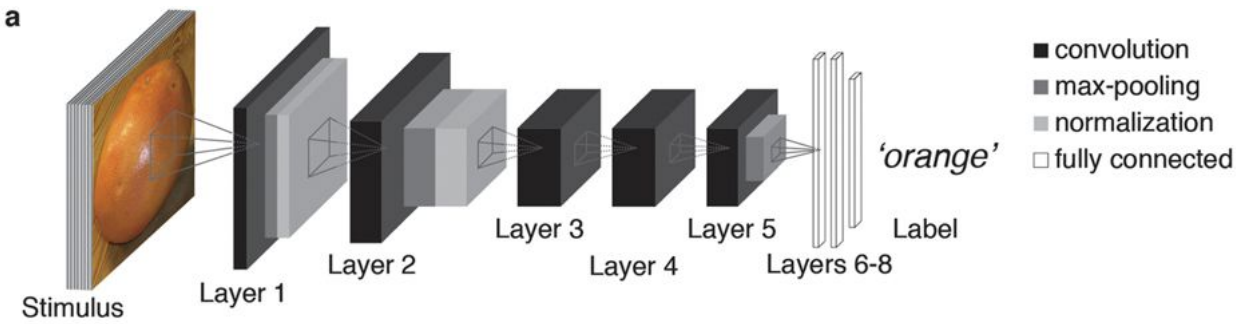
The architecture of deep convolutional neural networks was inspired by the ideas mentioned above

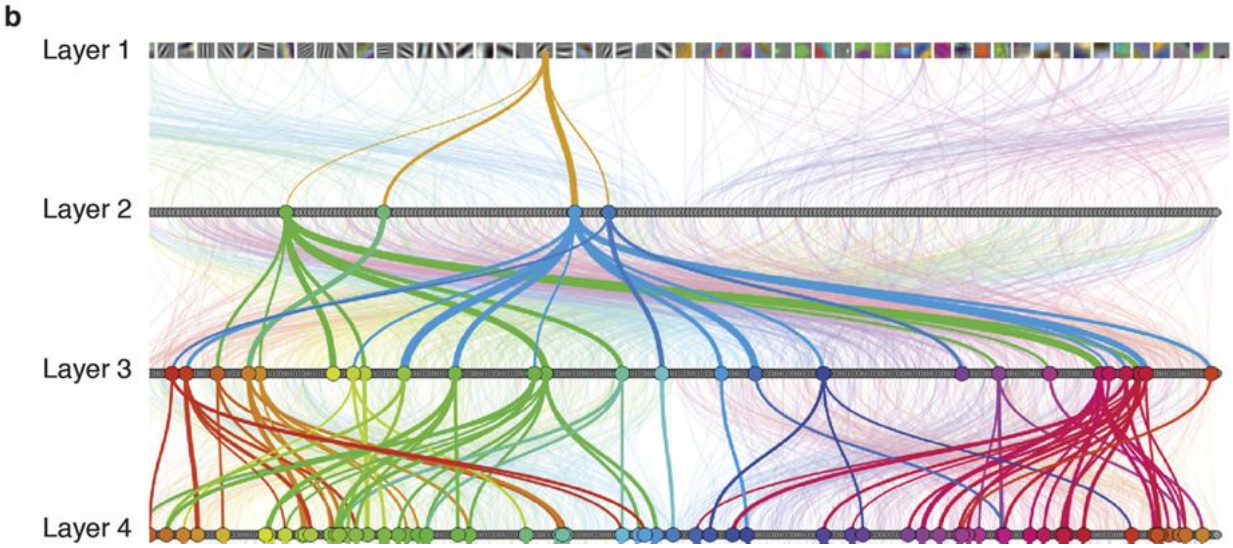
- local connections
- layering
- spatial invariance (shifting the input signal results in an equally shifted output signal. , most of us are able to recognize specific faces under a variety of conditions because we learn abstraction These abstractions are thus invariant to size, contrast, rotation, orientation

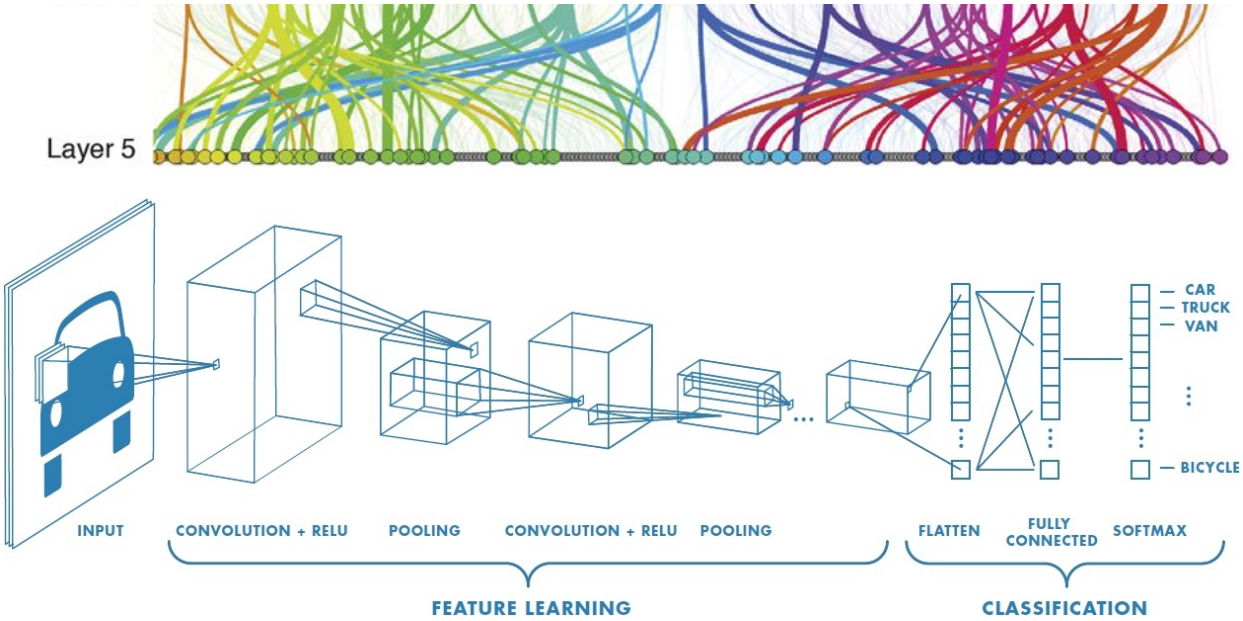
However, it remains to be seen if these computational mechanisms of convolutional neural networks are similar to the computation mechanisms occurring in the primate visual system

- convolution operation
- shared weights
- pooling/subsampling

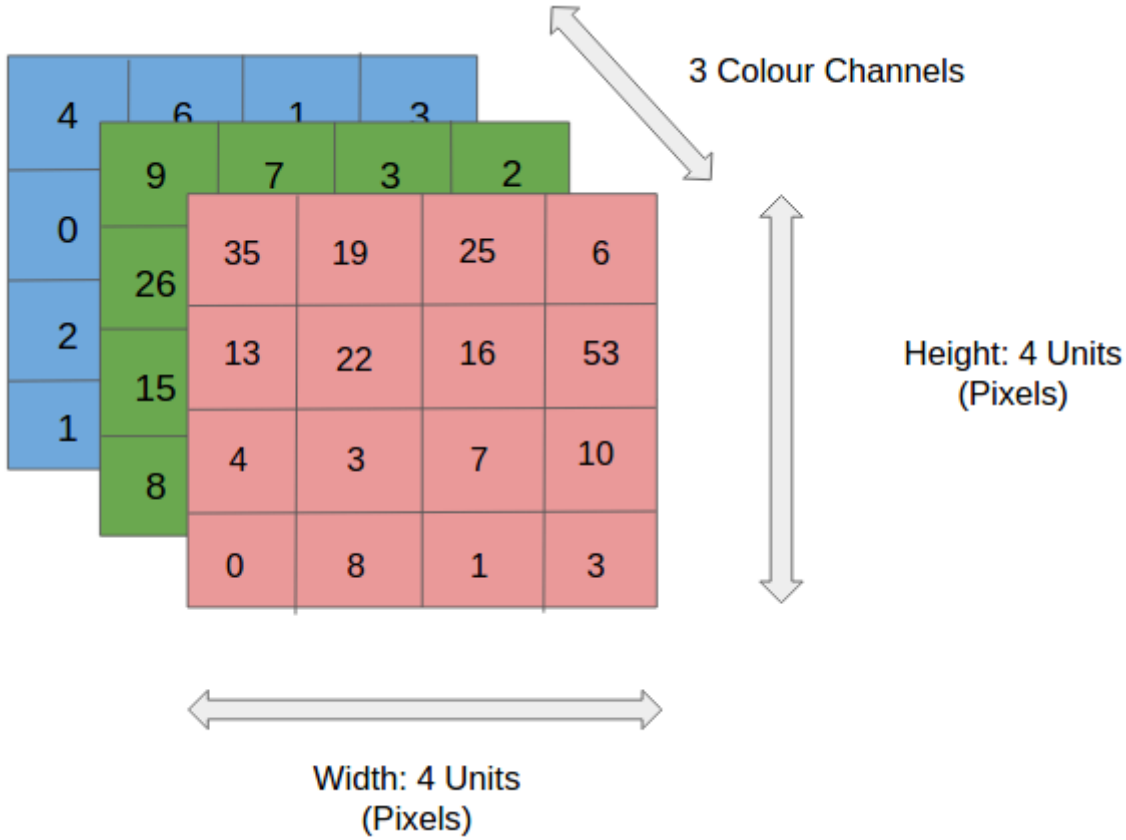
How does it work?







Step 1 - Prepare a dataset of images

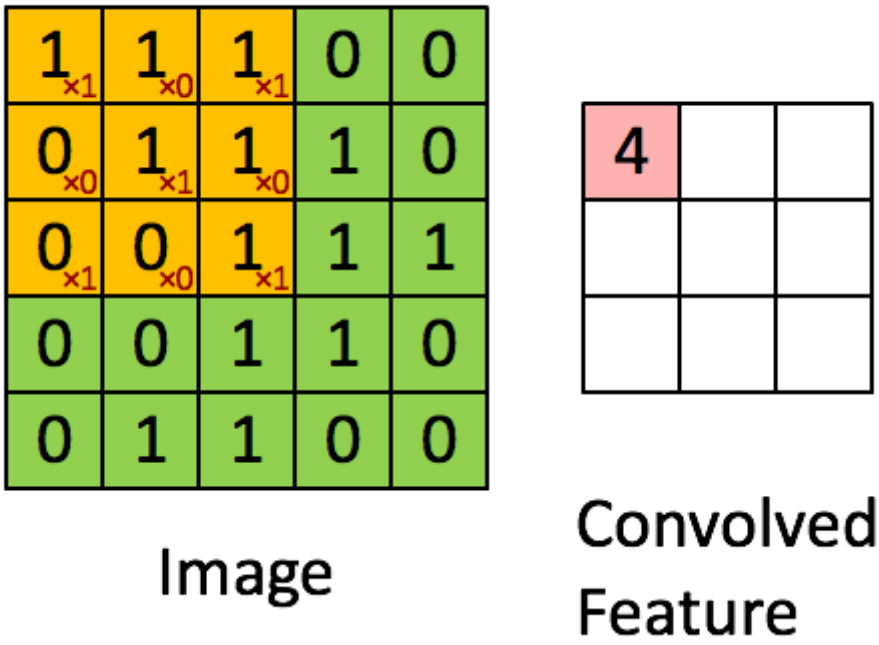


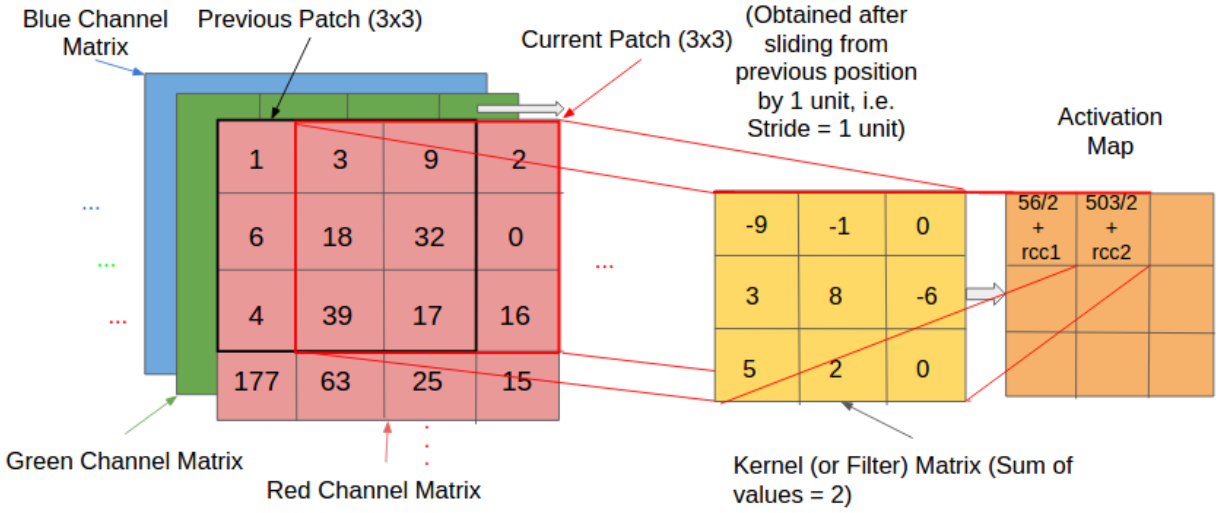
- Every image is a matrix of pixel values.
- The range of values that can be encoded in each pixel depends upon its bit size.
- Most commonly, we have 8 bit or 1 Byte-sized pixels. Thus the possible range of values a single pixel can represent is [0, 255].
- However, with coloured images, particularly RGB (Red, Green, Blue)-based images, the presence of separate colour channels (3 in the case of RGB images) introduces an additional 'depth' field to the data, making the input 3-dimensional.
- Hence, for a given RGB image of size, say 255×255 (Width x Height) pixels, we'll have 3 matrices associated with each image, one for each of the colour channels.
- Thus the image in it's entirety, constitutes a 3-dimensional structure called the Input Volume (255x255x3).

Great training datasets are CIFAR (https://www.cs.toronto.edu/~kriz/cifar.html) and CoCo

Great training datasets are CIFAR (https://www.cs.toronto.edu/~kriz/cifar.html) and CoCo (http://mscoco.org/). We'll use CIFAR.

Step 2 - Convolution

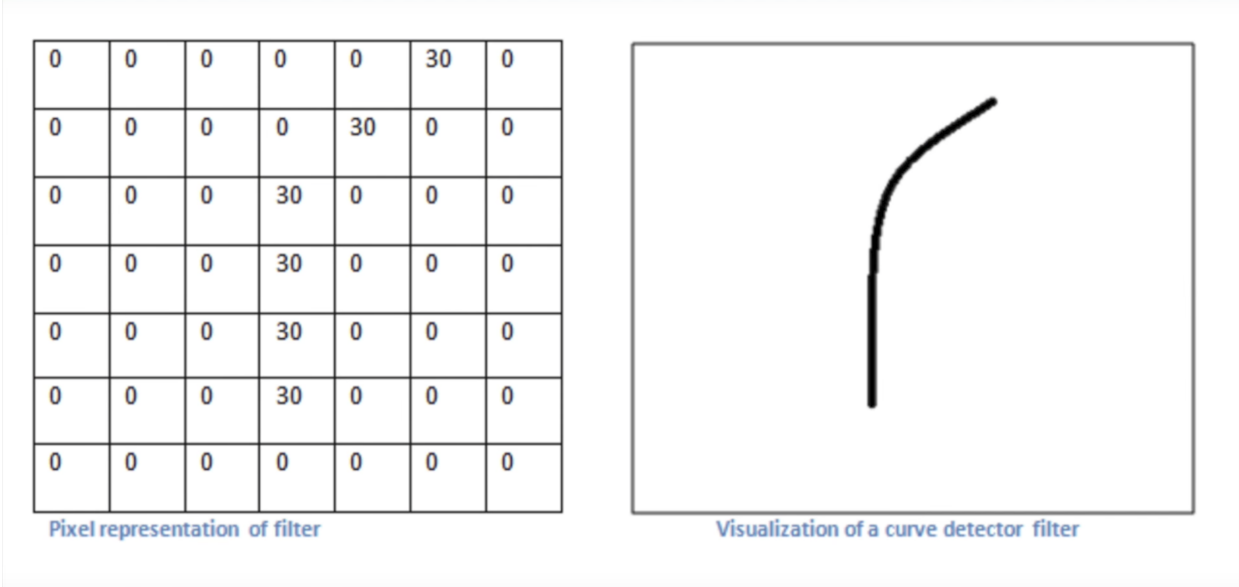


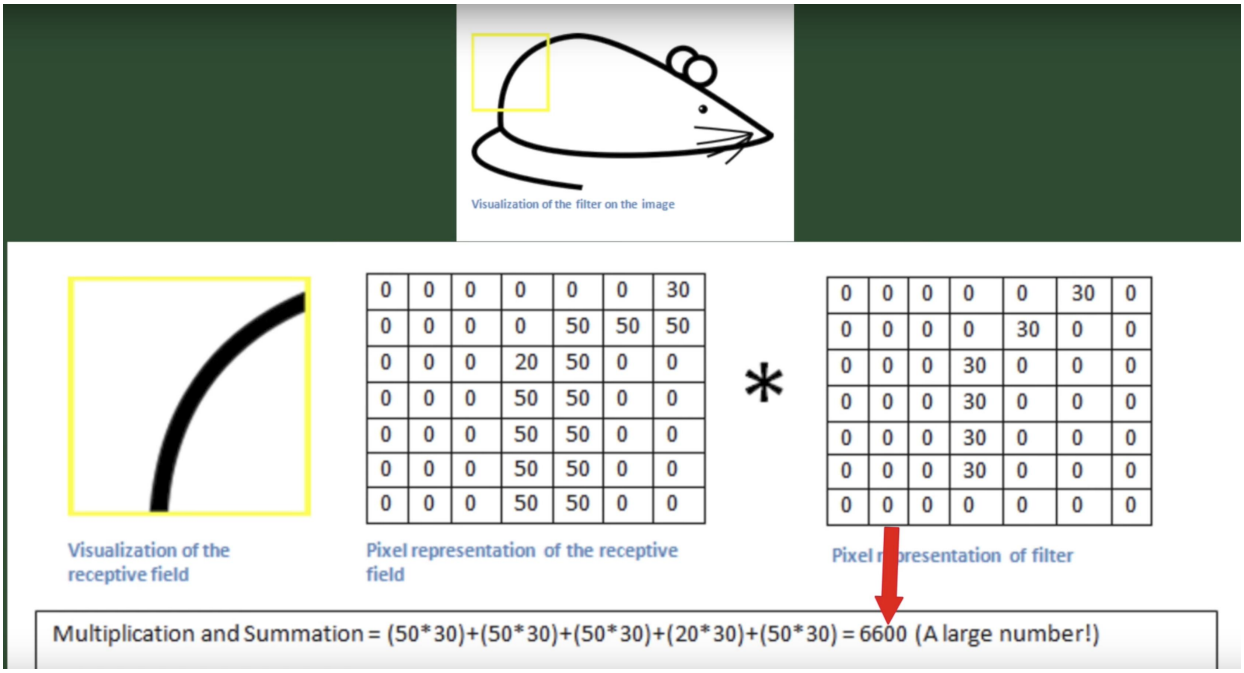


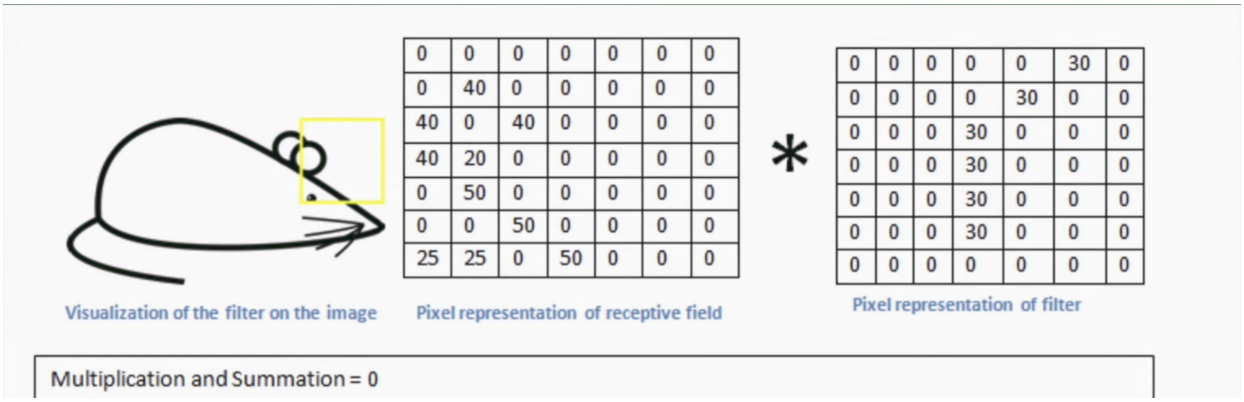
- A convolution is an orderly procedure where two sources of information are intertwined.
- A kernel (also called a filter) is a smaller-sized matrix in comparison to the input dimensions of the image, that consists of real valued entries.
- Kernels are then convolved with the input volume to obtain so-called 'activation maps' (also called feature maps).
- Activation maps indicate 'activated' regions, i.e. regions where features specific to the kernel have been detected in the input.
- The real values of the kernel matrix change with each learning iteration over the training set, indicating that the network is learning to identify which regions are of significance for extracting features from the data.
- We compute the dot product between the kernel and the input matrix. -The convolved value obtained by summing the resultant terms from the dot product forms a single entry in the activation matrix.
- The patch selection is then slided (towards the right, or downwards when the boundary of

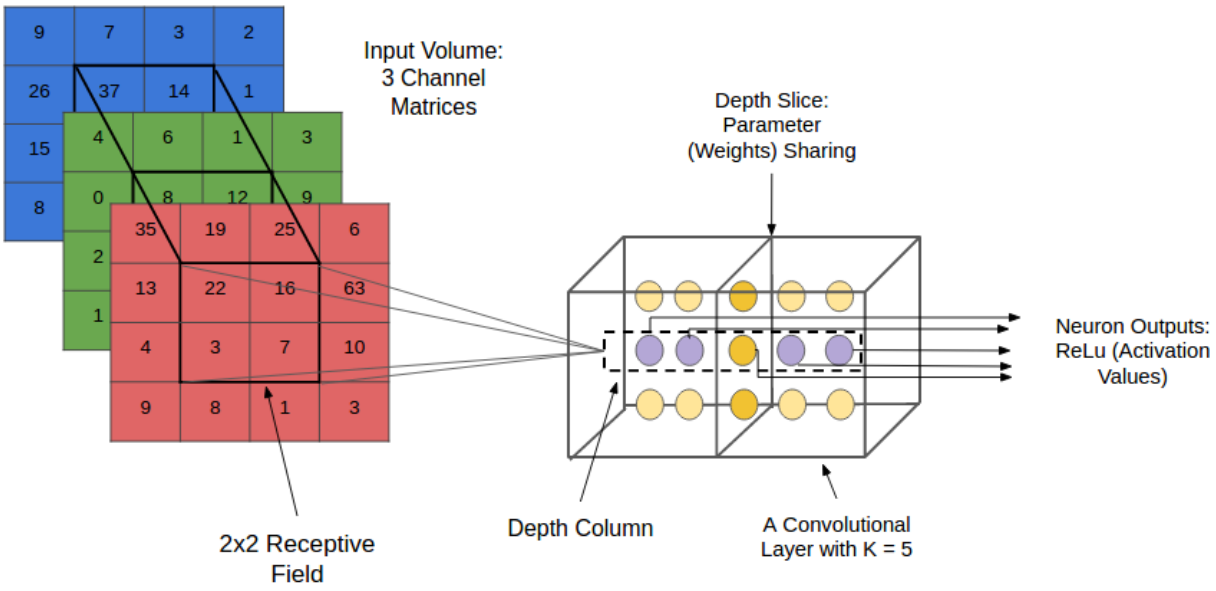
the matrix is reached) by a certain amount called the 'stride' value, and the process is repeated till the entire input image has been processed. - The process is carried out for all colour channels.

- instead of connecting each neuron to all possible pixels, we specify a 2 dimensional region called the 'receptive field[14]' (say of size 5×5 units) extending to the entire depth of the input (5x5x3 for a 3 colour channel input), within which the encompassed pixels are fully connected to the neural network's input layer. It's over these small regions that the network layer cross-sections (each consisting of several neurons (called 'depth columns')) operate and produce the activation map. (reduces computational complexity)





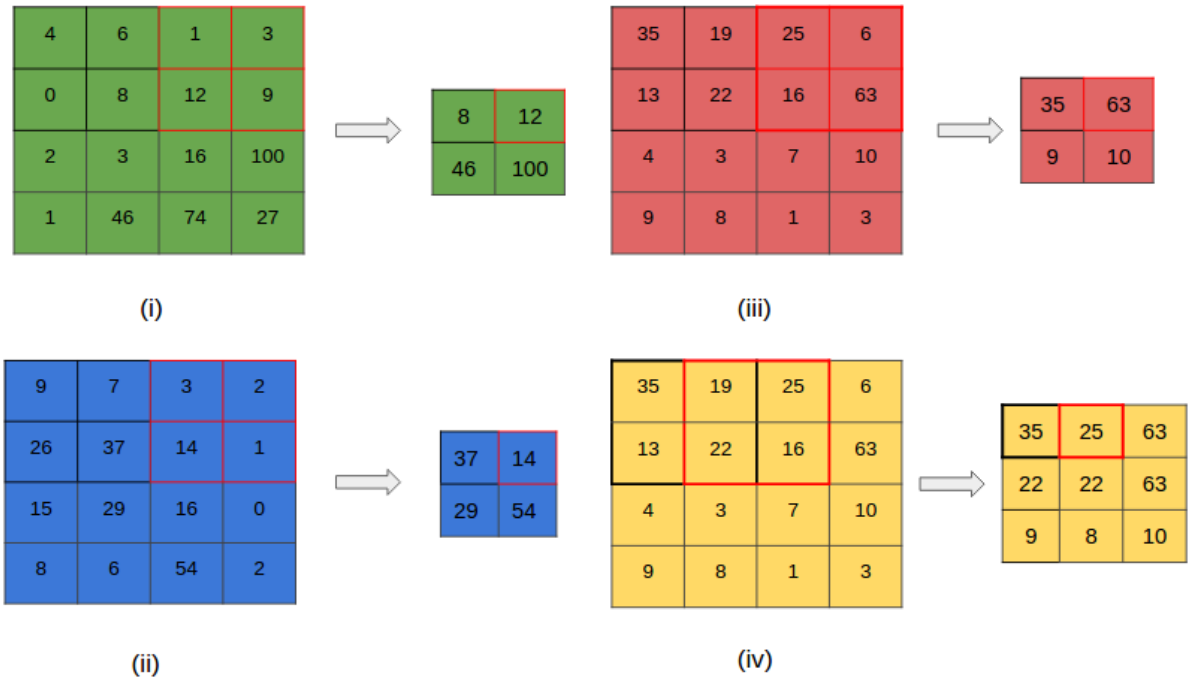




Great resource on description of convolution (discrete vs continous) & the fourier transform

http://timdettmers.com/2015/03/26/convolution-deep-learning/ (http://timdettmers.com/2015/03/26/convolution-deep-learning/)

Step 3 - Pooling



- Pooling reducing the spatial dimensions (Width x Height) of the Input Volume for the next Convolutional Layer. It does not affect the depth dimension of the Volume.
- The transformation is either performed by taking the maximum value from the values observable in the window (called 'max pooling'), or by taking the average of the values. Max pooling has been favoured over others due to its better performance characteristics.
- also called downsampling

Step 4 - Normalization (ReLU in our case)

$$Eq.3 : max(0, x)$$

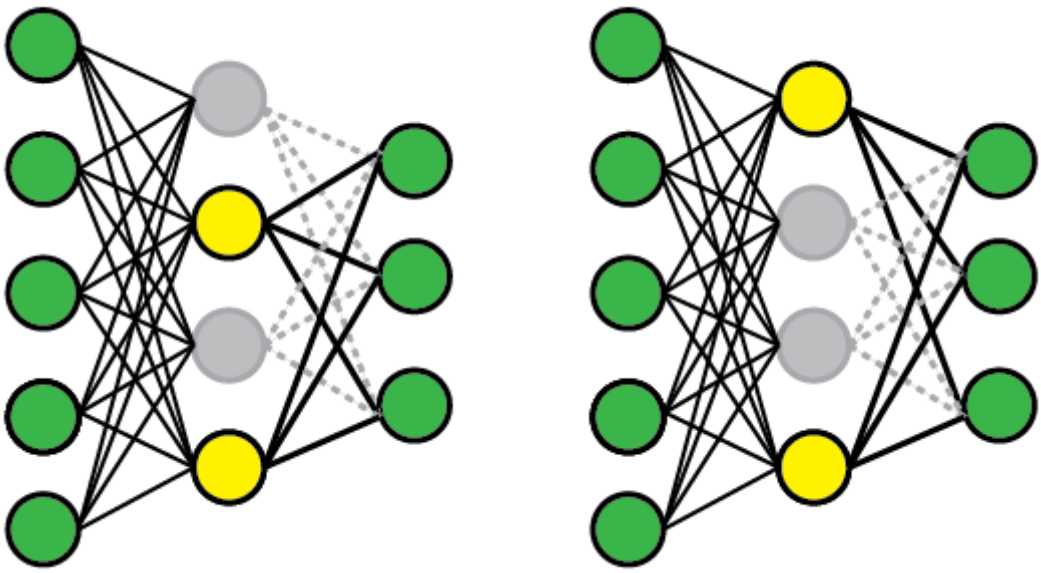
Normalization (keep the math from breaking by turning all negative numbers to 0) (RELU) a stack of images becomes a stack of images with no negative values.

Repeat Steps 2-4 several times. More, smaller images (feature maps created at every layer)

Step 5 - Regularization

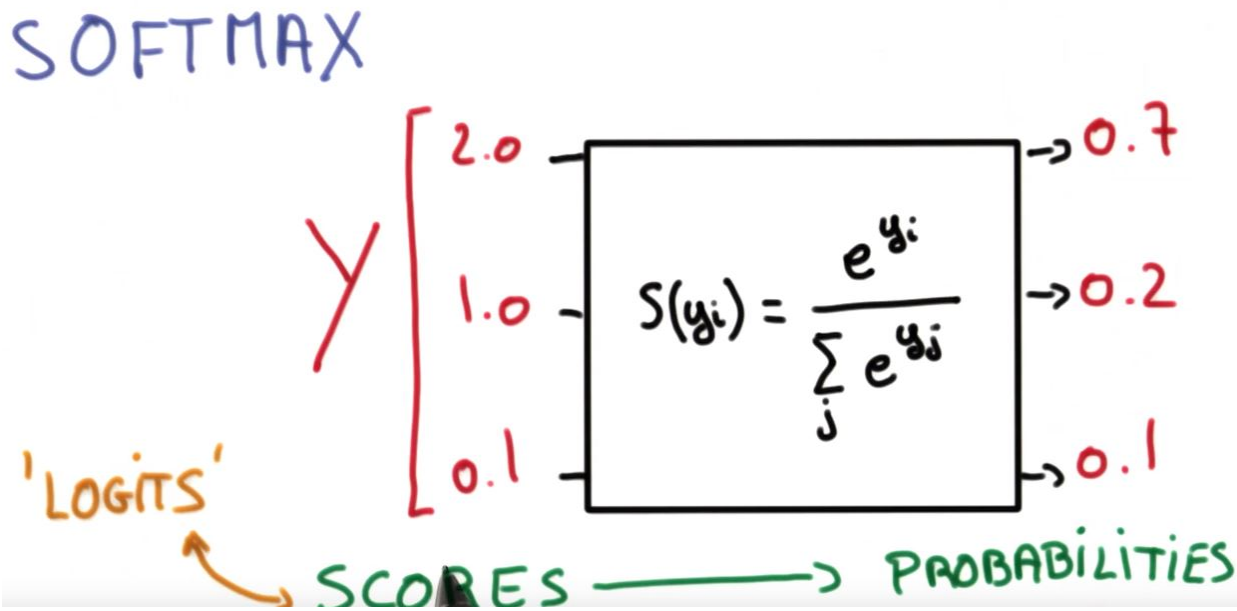
- Dropout forces an artificial neural network to learn multiple independent representations of the same data by alternately randomly disabling neurons in the learning phase.
- Dropout is a vital feature in almost every state-of-the-art neural network implementation.
- To perform dropout on a layer, you randomly set some of the layer's values to 0 during forward propagation.

See this (http://iamtrask.github.io/2015/07/28/dropout/)



Step 6 - Probability Conversion

At the very end of our network (the tail), we'll apply a softmax function to convert the outputs to probability values for each class.



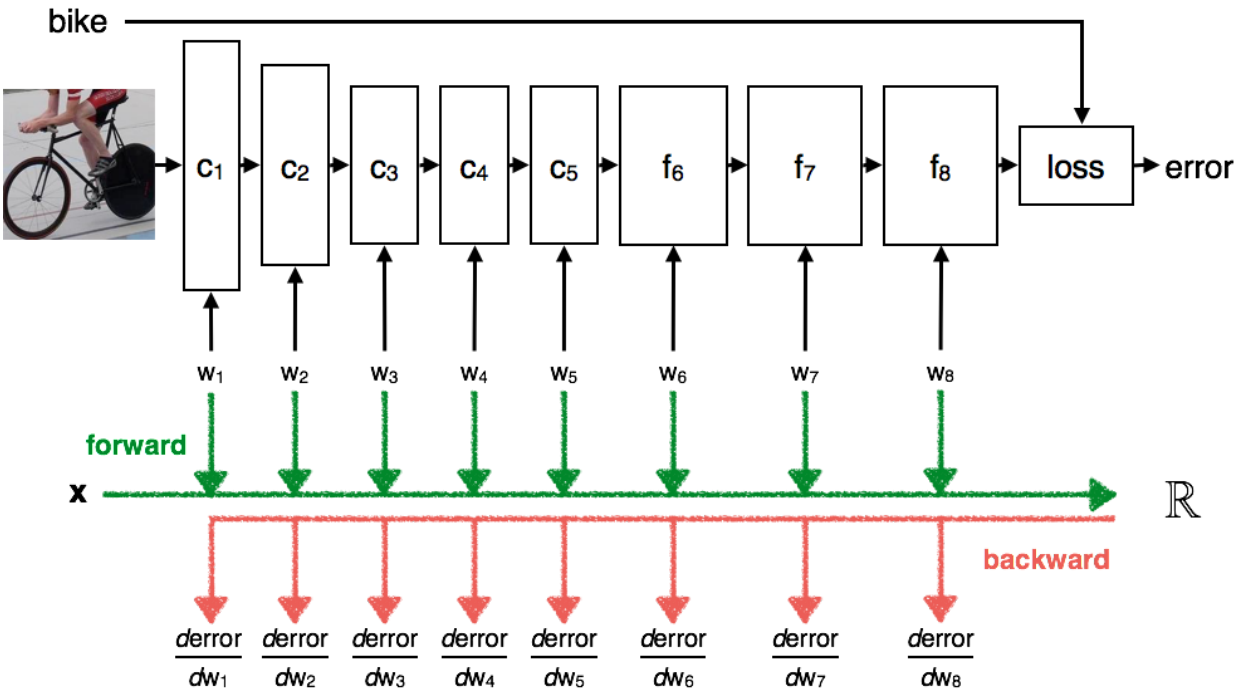
Step 7 - Choose most likely label (max probability value)

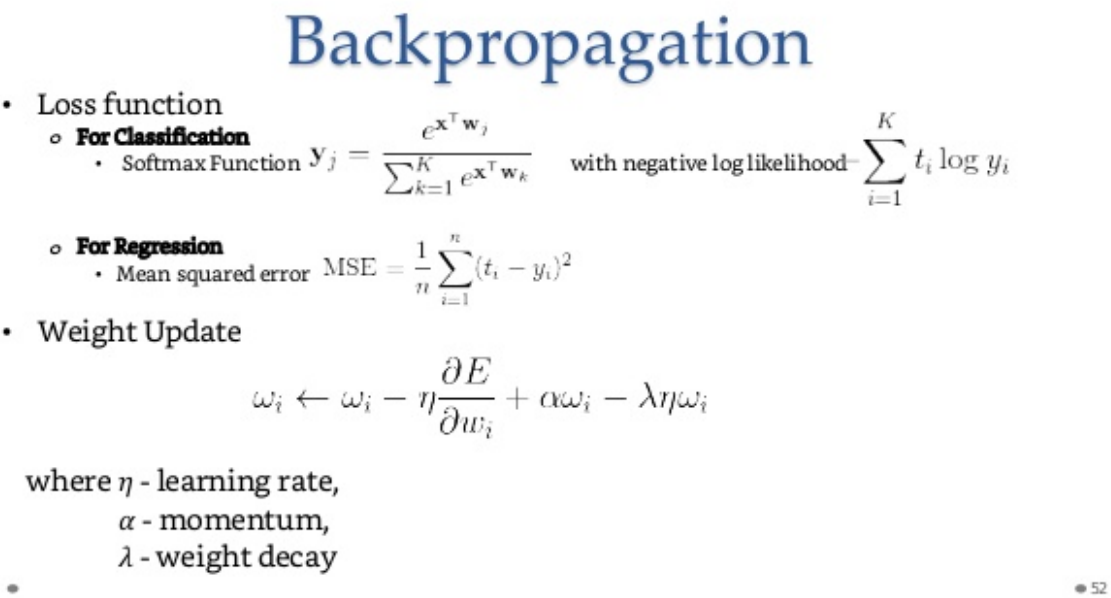
argmax(softmax_outputs)

These 7 steps are one forward pass through the network.

So how do we learn the magic numbers?

- We can learn features and weight values through backpropagation



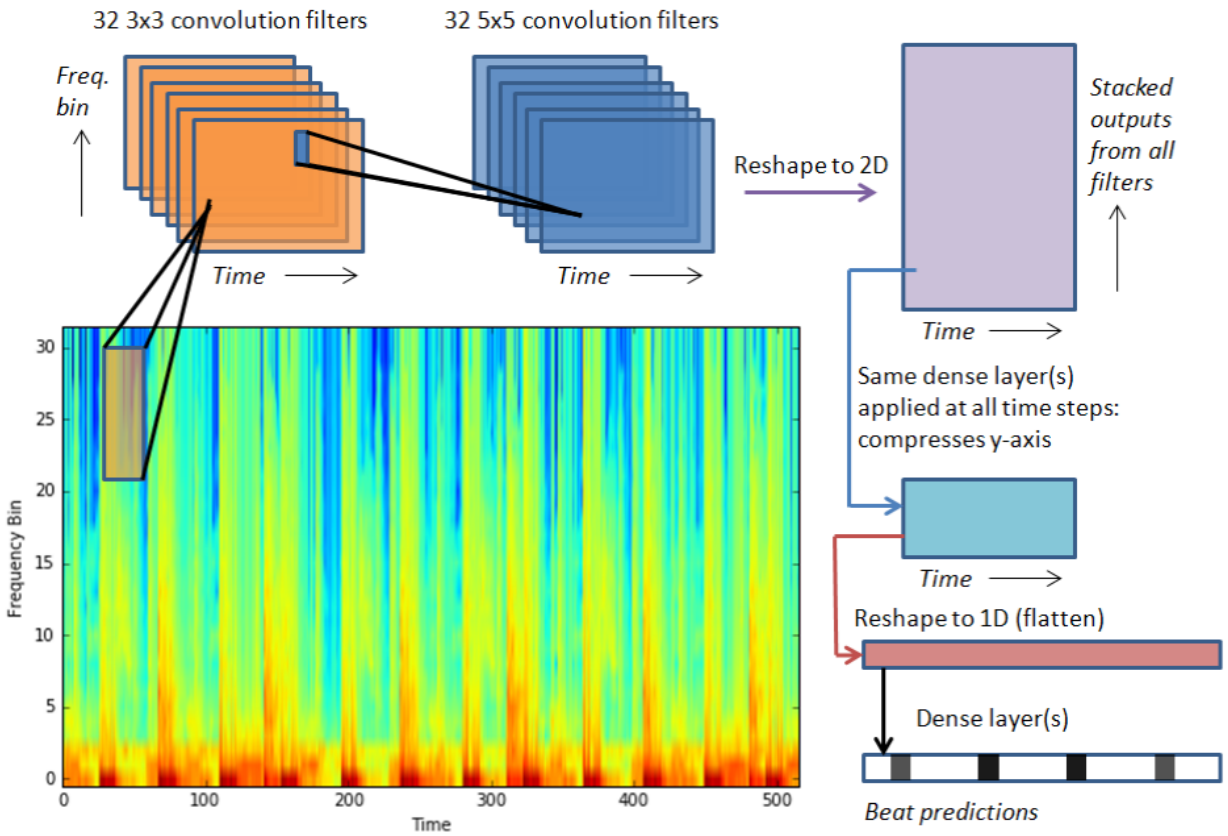


The other hyperparameters are set by humans and they are an active field of research (finding the optimal ones)

i.e - number of neurons, number of features, size of features, poooling window size, window stride

When is a good time to use it?

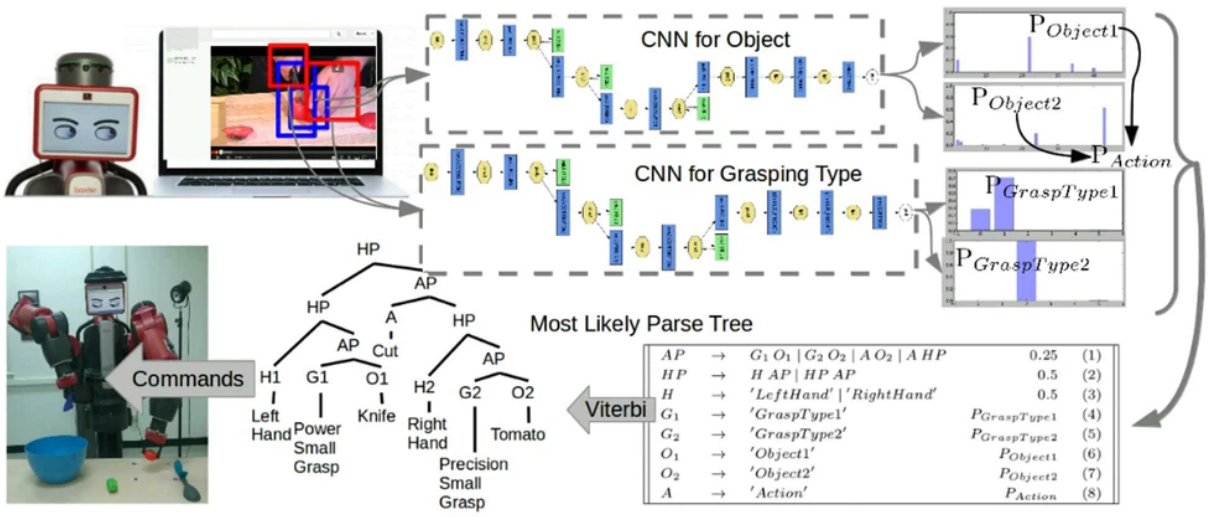
- To classify images
- To generate images (more on that later..)



But can also be applied to any any spatial 2D or 3D data. Images. Even sound and text. A rule of thumb is if you data is just as useful if you swap out the rows and columns, like customer data, then you can't use a CNN.

Good examples

Robot learns to grasp (combining CNNs)



Tensorflow! https://github.com/upul/CarND-TensorFlow-Lab (https://github.com/upul/CarND-TensorFlow-Lab)

Adversarial CNNs https://github.com/michbad/adversarial-mnist

Adversarial CNNs https://github.com/michbad/adversarial-mnist (https://github.com/michbad/adversarial-mnist)

In [ ]:
```
import pickle #saving and loading our serialized model
import numpy as np #matrix math
from app.model.preprocessor import Preprocessor as img_prep #image p
reprocessing

#class for loading our saved model and classifying new images
class LiteOCR:

        def __init__(self, fn="alpha_weights.pkl", pool_size=2):
        #load the weights from the pickle file and the meta data
                [weights, meta] = pickle.load(open(fn, 'rb'), encodi
ng='latin1') #currently, this class MUST be initialized from a pickl
e file
                #list to store labels
        self.vocab = meta["vocab"]

        #how many rows and columns in an image
                self.img_rows = meta["img_side"] ; self.img_cols = m
eta["img_side"]

        #load our CNN
                self.CNN = LiteCNN()
        #with our saved weights
                self.CNN.load_weights(weights)
        #define the pooling layers size
                self.CNN.pool_size=int(pool_size)

    #classify new image
        def predict(self, image):
                print(image.shape)
        #vectorize the image into the right shape for our network
                X = np.reshape(image, (1, 1, self.img_rows, self.img
_cols))
                X = X.astype("float32")

        #make the prediction
                predicted_i = self.CNN.predict(X)
        #return the predicted label
                return self.vocab[predicted_i]

class LiteCNN:
        def __init__(self):
        # a place to store the layers
                self.layers = []
        # size of pooling area for max pooling
                self.pool_size = None

        def load_weights(self, weights):
                assert not self.layers, "Weights can only be loaded
 once!"
        #add the saved matrix values to the convolutional network
                for k in range(len(weights.keys())):
                        self.layers.append(weights['layer_{}'.format
(k)])

        def predict(self, X):
        #here is where the network magic happens at a high level
        h = self.cnn_layer(X, layer_i=0, border_mode="full"); X= h
        h = self.relu_layer(X); X = h;
```

```
        h = self.cnn_layer(X, layer_i=2, border_mode="valid"); X = h
        h = self.relu_layer(X); X = h;
        h = self.maxpooling_layer(X); X = h
        h = self.dropout_layer(X, .25); X = h
        h = self.flatten_layer(X, layer_i=7); X = h;
        h = self.dense_layer(X, fully, layer_i=10); x = H
        h = self.softmax_layer2D(X); x = h
        max_i = self.classify(X)
        return max_i[0]

    #given our feature map we've learned from convolving around the
 image
    #lets make it more dense by performing pooling, specifically max
pooling
    #we'll select the max values from the image matrix and use that
 as our new feature map
        def maxpooling_layer(self, convolved_features):
        #given our learned features and images
                nb_features = convolved_features.shape[0]
                nb_images = convolved_features.shape[1]
                conv_dim = convolved_features.shape[2]
                res_dim = int(conv_dim / self.pool_size)       #assu
med square shape

        #initialize our more dense feature list as empty
                pooled_features = np.zeros((nb_features, nb_images,
res_dim, res_dim))
        #for each image
                for image_i in range(nb_images):
            #and each feature map
                        for feature_i in range(nb_features):
                #begin by the row
                                for pool_row in range(res_dim):
                    #define start and end points
                                        row_start = pool_row * self.
pool_size
                                        row_end   = row_start + self
.pool_size

                    #for each column (so its a 2D iteration)
                                        for pool_col in range(res_di
m):
                        #define start and end points
                                                col_start = pool_col
* self.pool_size
                                                col_end   = col_star
t + self.pool_size

                        #define a patch given our defined starting e
nding points
                                                patch = convolved_fe
atures[feature_i, image_i, row_start : row_end,col_start : col_end]
                        #then take the max value from that patch
                        #store it. this is our new learned feature/f
ilter
                                                pooled_features[feat
ure_i, image_i, pool_row, pool_col] = np.max(patch)
                return pooled_features

    #convolution is the most important of the matrix operations here
    #well define our input, lauyer number, and a border mode (explai
```

```
ned below)
        def cnn_layer(self, X, layer_i=0, border_mode = "full"):
        #we'll store our feature maps and bias value in these 2 vars
                features = self.layers[layer_i]["param_0"]
                bias = self.layers[layer_i]["param_1"]
        #how big is our filter/patch?
                patch_dim = features[0].shape[-1]
        #how many features do we have?
                nb_features = features.shape[0]
        #How big is our image?
                image_dim = X.shape[2] #assume image square
        #R G B values
                image_channels = X.shape[1]
        #how many images do we have?
                nb_images = X.shape[0]

        #With border mode "full" you get an output that is the "ful
l" size as the input.
        #That means that the filter has to go outside the bounds of
 the input by "filter size / 2" -
        #the area outside of the input is normally padded with zero
s.
                if border_mode == "full":
                        conv_dim = image_dim + patch_dim - 1
        #With border mode "valid" you get an output that is smaller
 than the input because
        #the convolution is only computed where the input and the fi
lter fully overlap.
                elif border_mode == "valid":
                        conv_dim = image_dim - patch_dim + 1

        #we'll initialize our feature matrix
                convolved_features = np.zeros((nb_images, nb_feature
s, conv_dim, conv_dim));
        #then we'll iterate through each image that we have
                for image_i in range(nb_images):
            #for each feature
                        for feature_i in range(nb_features):
                #lets initialize a convolved image as empty
                                convolved_image = np.zeros((conv_dim
, conv_dim))
                #then for each channel (r g b )
                                for channel in range(image_channels
):
                    #lets extract a feature from our feature map
                                        feature = features[feature_i
, channel, :, :]
                    #then define a channel specific part of our imag
e
                                        image   = X[image_i, channel
, :, :]
                    #perform convolution on our image, using a given
feature filter
                                        convolved_image += self.conv
olve2d(image, feature, border_mode);

                #add a bias to our convoved image
                                convolved_image = convolved_image +
bias[feature_i]
                #add it to our list of convolved features (learning
s)
                                convolved_features[image_i, feature
```

```
i, :, :] = convolved_image
                return convolved_features

    #In a dense layer, every node in the layer is connected to every
node in the preceding layer.
        def dense_layer(self, X, layer_i=0):
        #so we'll initialize our weight and bias for this layer
                W = self.layers[layer_i]["param_0"]
                b = self.layers[layer_i]["param_1"]
        #and multiply it by our input (dot product)
                output = np.dot(X, W) + b
                return output

        @staticmethod

    #so what does the convolution operation look like?, given an ima
ge and a feature map (filter)
        def convolve2d(image, feature, border_mode="full"):
        #we'll define the tensor dimensions of the image and the fea
ture
                image_dim = np.array(image.shape)
                feature_dim = np.array(feature.shape)
        #as well as a target dimension
                target_dim = image_dim + feature_dim - 1
        #then we'll perform a fast fourier transform on both the inp
ut and the filter
        #performing a convolution can be written as a for loop but f
or many convolutions
        #this approach is too comp. expensive/slow. it can be perfor
med orders of magnitude
        #faster using a fast fourier transform.
                fft_result = np.fft.fft2(image, target_dim) * np.fft
.fft2(feature, target_dim)
        #and set the result to our target
                target = np.fft.ifft2(fft_result).real

                if border_mode == "valid":
                        # To compute a valid shape, either np.all(x_
shape >= y_shape) or
                        # np.all(y_shape >= x_shape).
            #decide a target dimension to convolve around
                        valid_dim = image_dim - feature_dim + 1
                        if np.any(valid_dim < 1):
                                valid_dim = feature_dim - image_dim
+ 1
                        start_i = (target_dim - valid_dim) // 2
                        end_i = start_i + valid_dim
                        target = target[start_i[0]:end_i[0], start_i
[1]:end_i[1]]
                return target

        def relu_layer(x):
        #turn all negative values in a matrix into zeros
                z = np.zeros_like(x)
                return np.where(x>z,x,z)

        def softmax_layer2D(w):
        #this function will calculate the probabilities of each
        #target class over all possible target classes.
                maxes = np.amax(w, axis=1)
                maxes = maxes.reshape(maxes.shape[0], 1)
```

```
                e = np.exp(w - maxes)
                dist = e / np.sum(e, axis=1, keepdims=True)
                return dist

    #affect the probability a node will be turned off by multiplying
it
    #by a p values (.25 we define)
        def dropout_layer(X, p):
                retain_prob = 1. - p
                X *= retain_prob
                return X

    #get the largest probabililty value from the list
        def classify(X):
                return X.argmax(axis=-1)

    #tensor transformation, less dimensions
        def flatten_layer(X):
                flatX = np.zeros((X.shape[0],np.prod(X.shape[1:])))
                for i in range(X.shape[0]):
                        flatX[i,:] = X[i].flatten(order='C')
                return flatX
```